

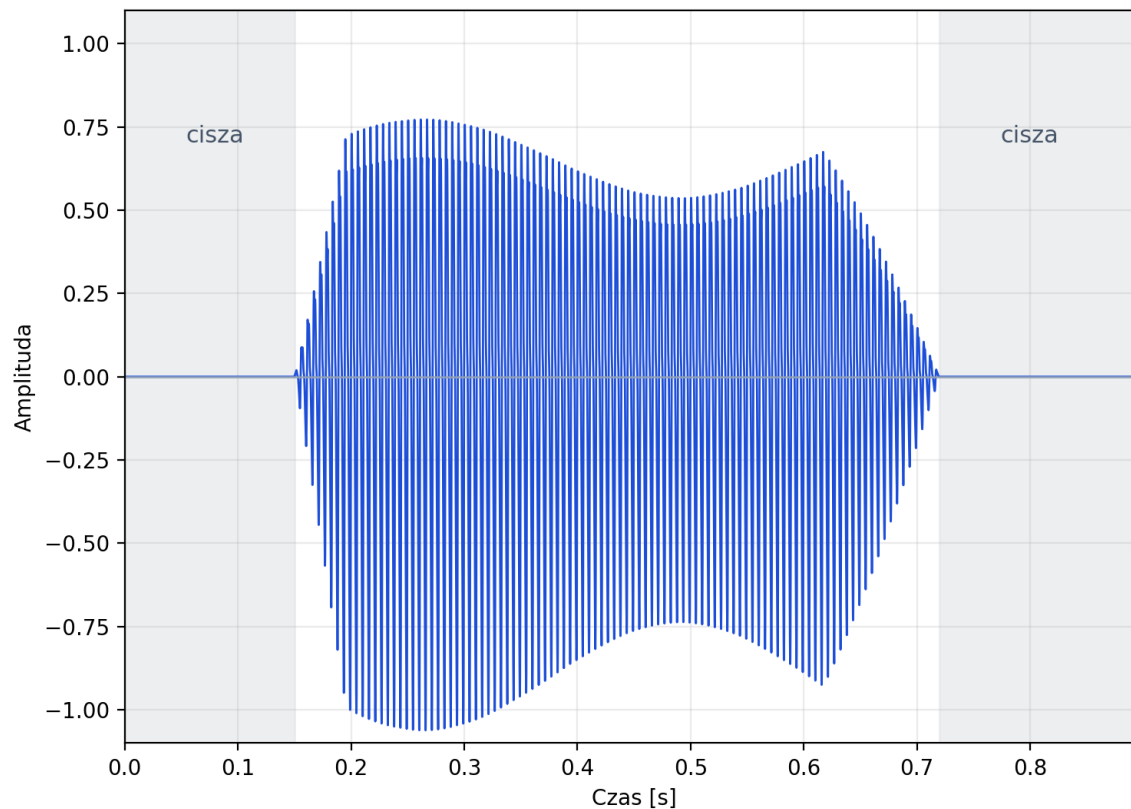
Od sygnałów do cech i modeli

Iwona Christop

16 maja 2026 r.

Jaką informację powinniśmy zachować z dźwięku, aby model mógł skutecznie uczyć się z audio?

- Nie wszystko w sygnale jest równie ważne
- Różne zadania potrzebują różnych reprezentacji.
- Dobra reprezentacja może bardzo ułatwić uczenie.



Co model może wydobyć z takiej fali?

ton podstawowy

Najniższa dominująca częstotliwość,
czyli główna wysokość dźwięku

harmoniczne

Dodatkowe składowe przy wielokrotnościach
częstotliwości podstawowej

energia

Jak silny jest sygnał w danym fragmencie
czasu

cisza

Fragmenty bez istotnego sygnału,
np. przed i po słowie

Cecha to mierzalna własność danych, która pomaga modelowi zauważać użyteczne wzorce.

Na przykład:

- energia sygnału,
- wysokość dźwięku lub jej przybliżenie,
- kształt widma,
- relacje między pasmami częstotliwości.

Jakie informacje mogą stać się cechami?

Czas

- energia
- obwiednia
- tempo zmian

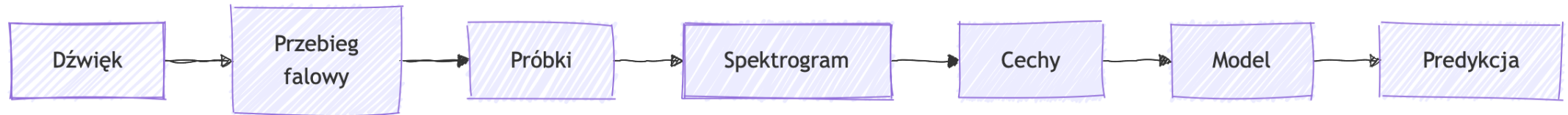
Częstotliwość

- piki widma
- nachylenie widma
- rozkład energii

Reprezentacje uczone

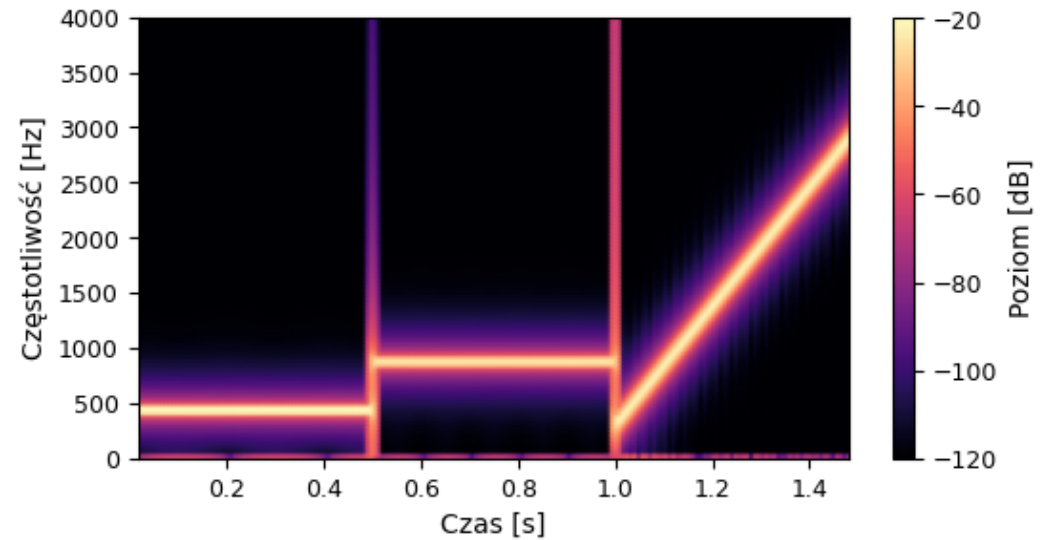
- embeddingi
- wektory ukryte
- reprezentacje kontekstowe

Typowy potok audio



Dlaczego spektrogram jest punktem zwrotnym?

- Łączy czas i częstotliwość.
- Pokazuje, kiedy pojawia się dana energia.
- Jest znacznie czytelniejszy dla wielu zadań audio niż sam przebieg.

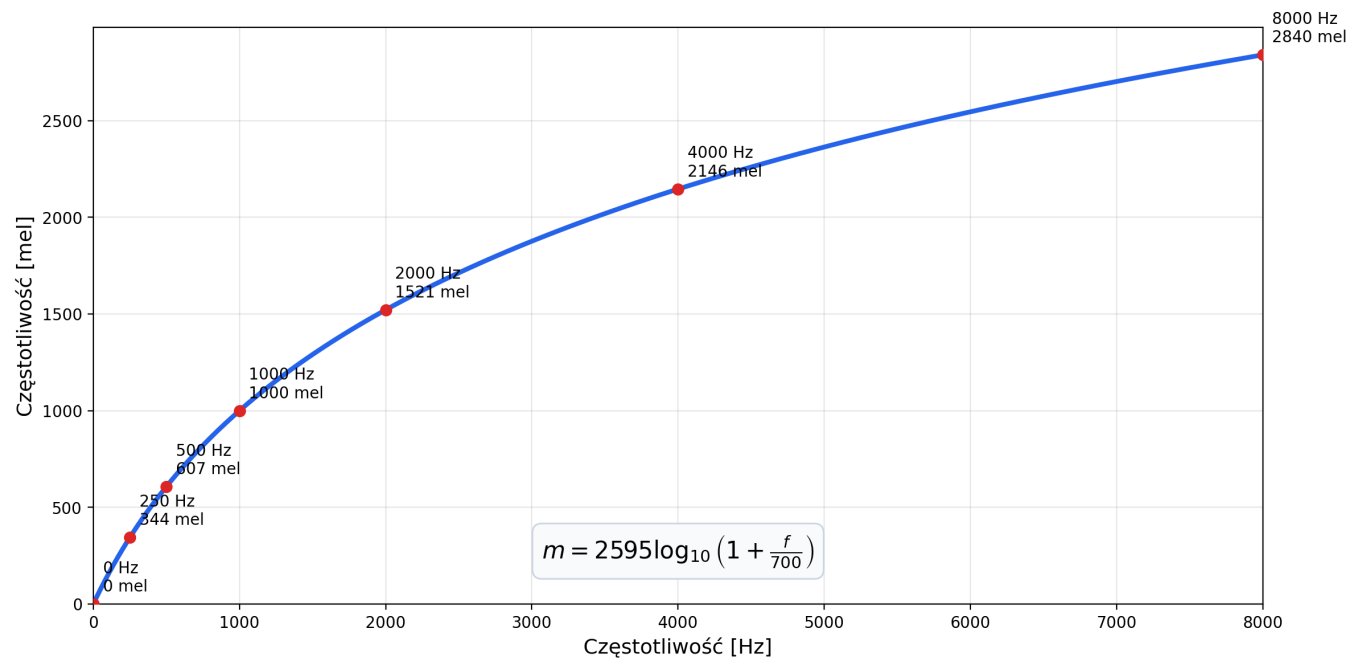


Skala melowa to skala częstotliwości zaprojektowana tak, aby lepiej odpowiadała temu, jak ludzie odczuwają różnice wysokości dźwięku.

Po co zmieniać oś częstotliwości?

- przy niskich częstotliwościach słyszymy mniejsze różnice bardziej wyraźnie,
- przy wysokich częstotliwościach często wystarcza bardziej zgrubny podział,
- dlatego użyteczne jest nierówne grupowanie pasm.

Konwersja z herców na mele



Spektrogram melowy to spektrogram, w którym:

- oś częstotliwości zastępujemy pasmami melowymi,
- sąsiednie częstotliwości są łączone w szersze grupy,
- otrzymujemy krótszy i bardziej uporządkowany opis.



Dlaczego po melu często bierzemy logarytm?

- energia w audio może mieć bardzo duży zakres wartości,
- logarytm zmniejsza przewagę bardzo dużych wartości,
- słabsze składowe stają się bardziej widoczne,
- taki opis bywa stabilniejszy dla modeli.

Różnica między 10 a 100 jest duża, ale między 10 000 a 10 090 zwykle nie chcemy traktować jej jako równie ważnej. Logarytm pomaga takie zakresy uporządkować.

MFCC, czyli *Mel-Frequency Cepstral Coefficients*, są zwartym opisem kształtu krótkoczasowego widma, zwykle obliczanym z logarytmicznego widma melowego.

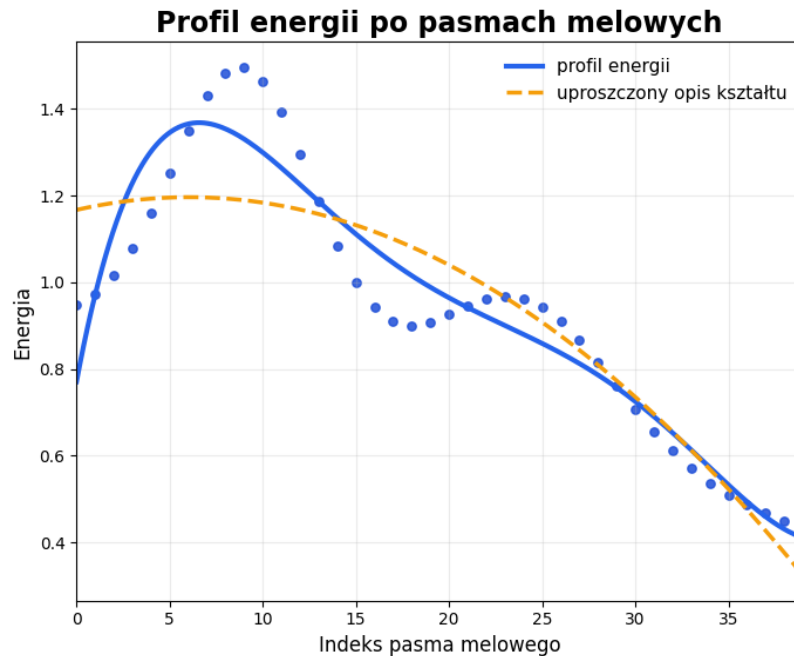
Są to klasyczne cechy audio, szczególnie popularne w mowie. Są znacznie krótsze niż pełny spektrogram i dobrze opisują ogólny kształt widma.

$$\text{MFCC} = \text{DCT}(\log(\text{widmo melowe}))$$

1. Zaczynamy od krótkiego fragmentu sygnału.
2. Liczymy jego widmo lub spektrogram.
3. Grupujemy energię w pasma melowe.
4. Bierzemy logarytm.
5. Stosujemy DCT i dostajemy krótki wektor.

Co daje DCT w MFCC?

- zamienia szczegółowy profil pasm na współczynniki opisujące jego kształt,
- pierwsze współczynniki przechowują najbardziej ogólną informację,
- wiele dalszych współczynników można pominąć bez ogromnej straty.



Kilka współczynników zamiast całego konturu

Poziom średni
0.944

Nachylenie
-0.0216

Krzywizna
-0.0008

*Zamiast zapamiętywać cały szczegółowy kontur,
zapisujemy jego główne cechy kształtu.*

MFCC pomagają, gdy

- chcemy mieć krótki wektor,
- pracujemy z mową lub prostszym modelem,
- zależy nam na klasycznych, sprawdzonych cechach.

Ale mogą ograniczać, gdy

- potrzebne są bardzo drobne szczegóły widma,
- zadanie dotyczy złożonej muzyki lub bogatego otoczenia,
- nowoczesna sieć może nauczyć się lepszej reprezentacji sama.

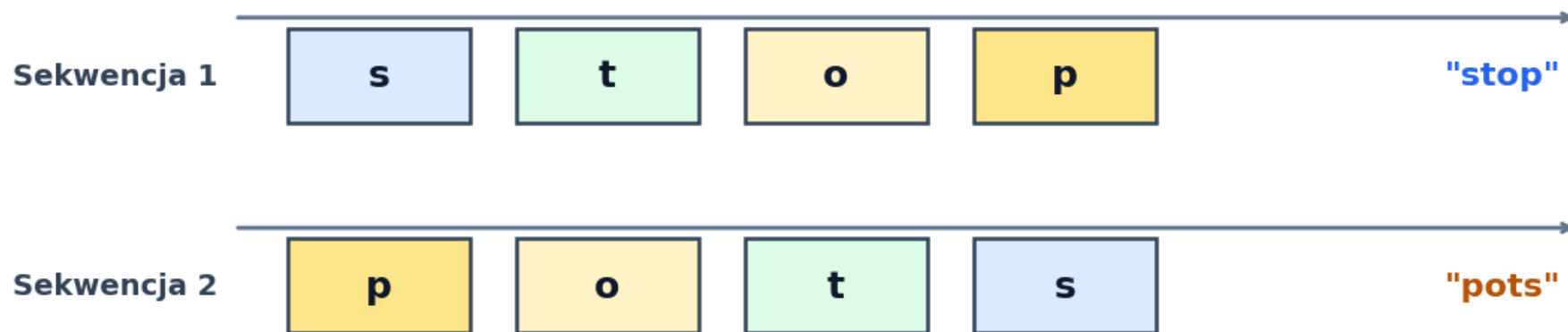
Fonem to najmniejsza jednostka dźwiękowa, która może zmieniać znaczenie słowa.

Przykłady:

- /b/ i /p/ odróżniają "bat" od "pat",
- /m/ i /n/ odróżniają "mat" od "nat".

Kolejność w czasie ma znaczenie

- te same dźwięki w innej kolejności mogą oznaczać coś innego,
- przejścia między fonemami są równie ważne jak same fonemy,
- model mowy musi uwzględniać wzorce czasowe.



Ekstrakcja cech w praktyce

```
ramka 1 -> wektor cech  
ramka 2 -> wektor cech  
ramka 3 -> wektor cech  
...
```

- Długie nagranie dzielimy zwykle na krótkie okna czasu.
- Z każdego okna obliczamy zestaw liczb.
- Wynik to sekwencja wektorów albo macierz cech.

Reprezentacja to forma, w jakiej informacja o dźwięku jest przedstawiona modelowi lub wewnątrz modelu.

Przykłady reprezentacji:

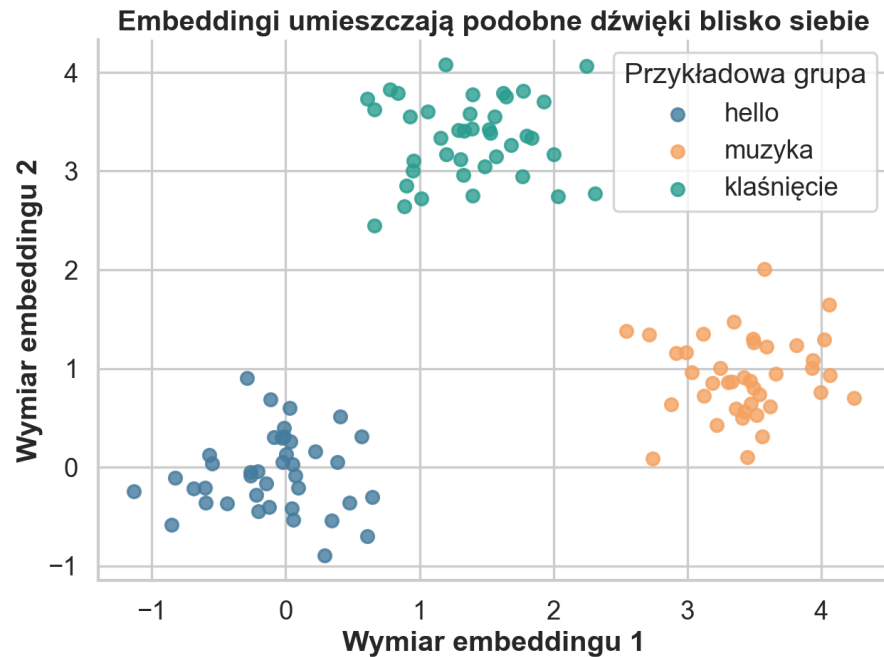
- surowe próbki,
- spektrogram,
- MFCC,
- wektor ukryty w sieci neuronowej.

Embedding to wektor nauczony tak, aby podobne dane wejściowe miały podobne reprezentacje liczbowe.

$$z \in \mathbb{R}^d$$

gdzie z to wektor embeddingu, a d to liczba wymiarów.

Każdy wymiar jest jedną liczbą, ale znaczenie powstaje z całego wektora.



W dobrej przestrzeni embeddingów:

- podobne przykłady leżą blisko,
- różne klasy są od siebie oddzielone,
- proste modele mogą łatwiej podjąć decyzję.

Ręczne cechy

- człowiek decyduje, co mierzyć,
- łatwiej je interpretować,
- dobrze działają w klasycznych pipeline'ach.

Uczenie reprezentacji

- model sam odkrywa użyteczne wzorce,
- wymaga zwykle większej ilości danych,
- może znaleźć lepsze cechy niż ręcznie projektowane.

Uczenie samonadzorowane to sposób uczenia, w którym same dane dostarczają części sygnału uczącego, na przykład przez maskowanie, przewidywanie albo porównywanie fragmentów wejścia.

Dzięki temu nie trzeba ręcznie opisać każdego nagrania etykietą i można uczyć się z dużych zbiorów surowego audio.

Celem jest nauczenie dobrej reprezentacji przed zadaniem końcowym.

Dwa style budowania systemów audio

Klasyczny pipeline

- audio
- cechy ręczne
- mały model
- predykcja

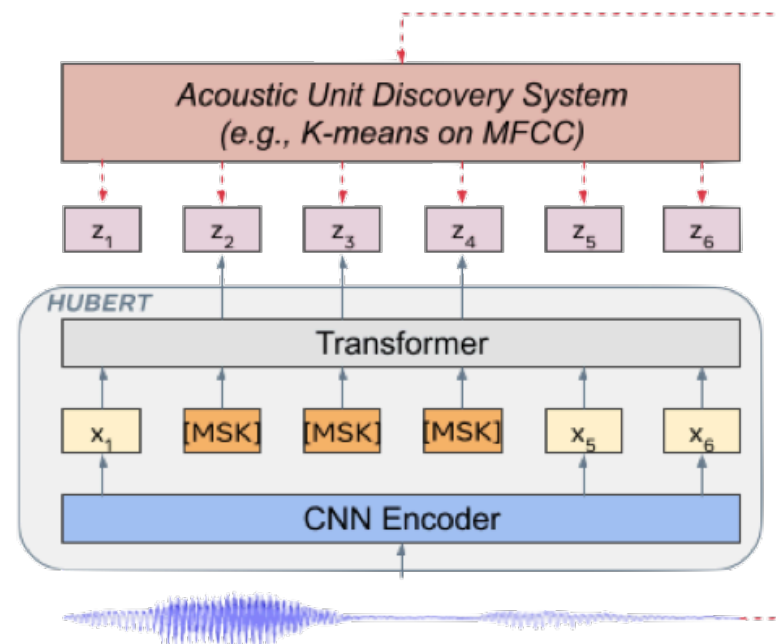
Pipeline uczony

- audio
- duży model uczący reprezentacji
- predykcja

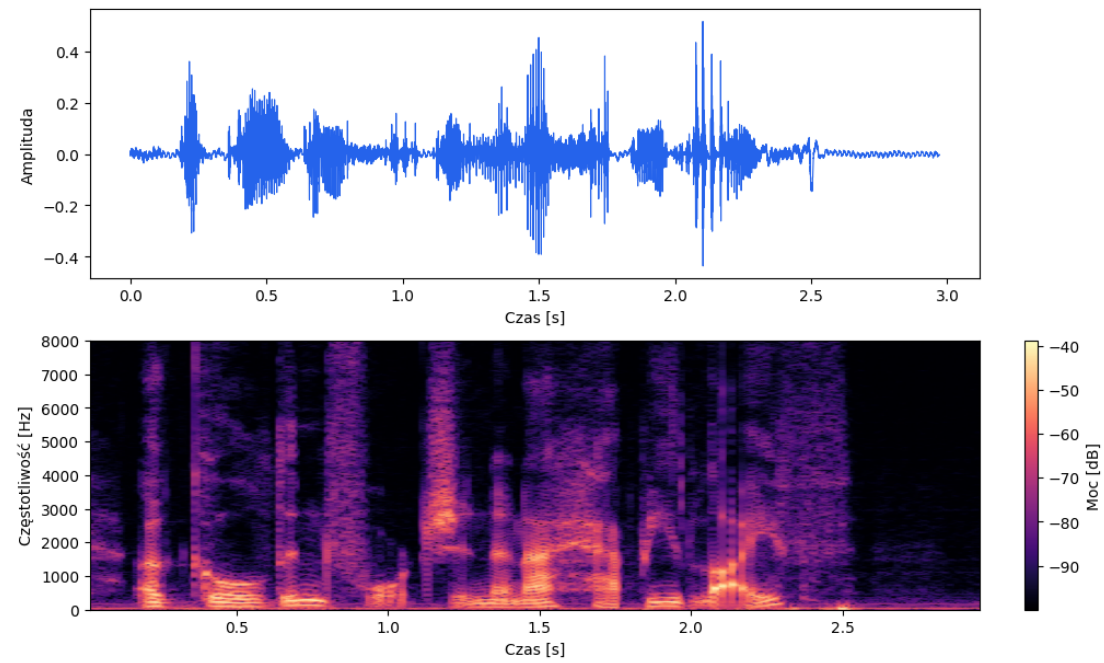
HuBERT

HuBERT (*Hidden-Unit BERT*) to model do **uczenia reprezentacji mowy**, a nie od razu do wypisywania tekstu. W uczeniu wstępnym przewiduje etykiety ukrytych jednostek dla **zamaskowanych** fragmentów audio.

1. Wejście to surowe audio (16 kHz).
2. Warstwy konwolucyjne zamieniają falę na sekwencję ramek cech.
3. Część ramek jest maskowana.
4. Transformer encoder buduje reprezentację kontekstową.
5. Głowica predykcyjna odtwarza ukryte jednostki akustyczne.



Przykład



HuBERT: transkrypcja

```
import torch, transformers

# audio -> wejście modelu
processor = transformers.Wav2Vec2Processor.from_pretrained("facebook/hubert-large-ls960-ft")
model = transformers.HubertForCTC.from_pretrained("facebook/hubert-large-ls960-ft").eval()

# próbki -> tensor wejściowy
input_values = processor(audio, sampling_rate=sr, return_tensors="pt").input_values

# wyłączamy gradienty dla szybkości
with torch.inference_mode():
    # model liczy wyniki dla kolejnych ramek
    logits = model(input_values).logits

# wybieramy najlepszy token w każdym kroku
predicted_ids = torch.argmax(logits, dim=-1)

# tokeny zamieniamy na tekst
transcription = processor.decode(predicted_ids[0])
```

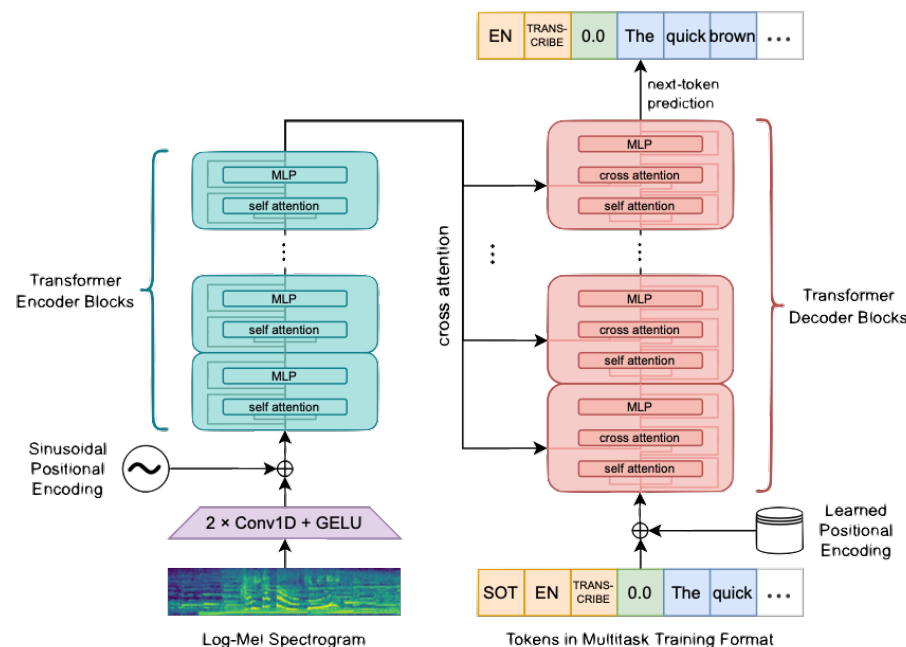
Python

```
input_values.shape = (1, 47520)
transcription = A GOLDEN FORTUNE AND A HAPPY LIFE
```

Whisper

Whisper to duży model **end-to-end** do rozpoznawania i tłumaczenia mowy. Wejściem nie jest surowa fala, tylko **80-kanalowy log-mel spektrogram**. Model przewiduje tekst autoregresyjnie, token po tokenie.

1. Audio jest dzielone na okna do 30 sekund.
2. Z audio liczymy log-mel spektrogram.
3. **Warstwy konwolucyjne** przygotowują sekwencję dla enkodera.
4. **Transformer encoder** buduje reprezentację akustyczną.
5. **Transformer decoder** z cross-attention generuje tekst.



Whisper

```
import torch, transformers

# audio -> log-mel i tokeny
processor = transformers.AutoProcessor.from_pretrained("openai/whisper-small")
model = transformers.WhisperForConditionalGeneration.from_pretrained("openai/whisper-small").eval()

# próbki -> cechy log-mel
input_features = processor(audio, sampling_rate=sr, return_tensors="pt").input_features

# wyłączamy gradienty
with torch.inference_mode():
    # model generuje tekst
    generated_ids = model.generate(input_features=input_features, max_new_tokens=64)

# tokeny -> tekst
transcription = processor.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

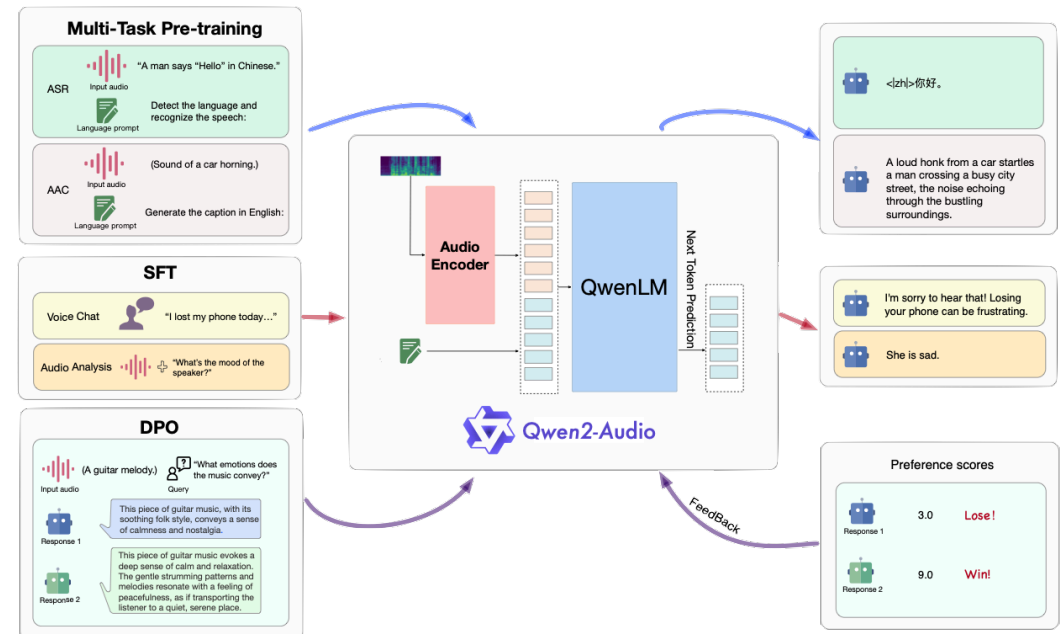
Python

```
input_features.shape = (1, 80, 3000)
transcription = A golden fortune and a happy life.
```

Qwen2-Audio

Qwen2-Audio to model typu **audio-language model**. Przyjmuje **audio oraz tekstowe instrukcje** i zwraca odpowiedź tekstową. Nie służy tylko do ASR, ale też do opisu dźwięków i odpowiadania na pytania o nagranie.

1. Wejście audio jest zamieniane na reprezentację melową.
2. **Audio encoder** analizuje sekwencję ramek.
3. Adapter lub projector mapuje reprezentację audio do przestrzeni modelu językowego.
4. **Decoder-only language model Qwen** generuje odpowiedź tekstową.



Qwen2-Audio

```
import torch, transformers

processor = transformers.AutoProcessor.from_pretrained("Qwen/Qwen2-Audio-7B")
model = transformers.Qwen2AudioForConditionalGeneration.from_pretrained("Qwen/Qwen2-Audio-7B").eval()

# budujemy wejście z audio i instrukcji
inputs = processor(
    text="<|AUDIO|>Transcribe this speech exactly.",      # mówimy modelowi, czego chcemy
    audio=audio,                                           # przekazujemy nasze nagranie
    return_tensors="pt",
)

# uruchamiamy model tylko do odczytu
with torch.inference_mode():
    # model dopisuje odpowiedź
    generated_ids = model.generate(**inputs, max_new_tokens=64)

# odcinamy część promptu
new_tokens = generated_ids[:, inputs["input_ids"].shape[1] :]

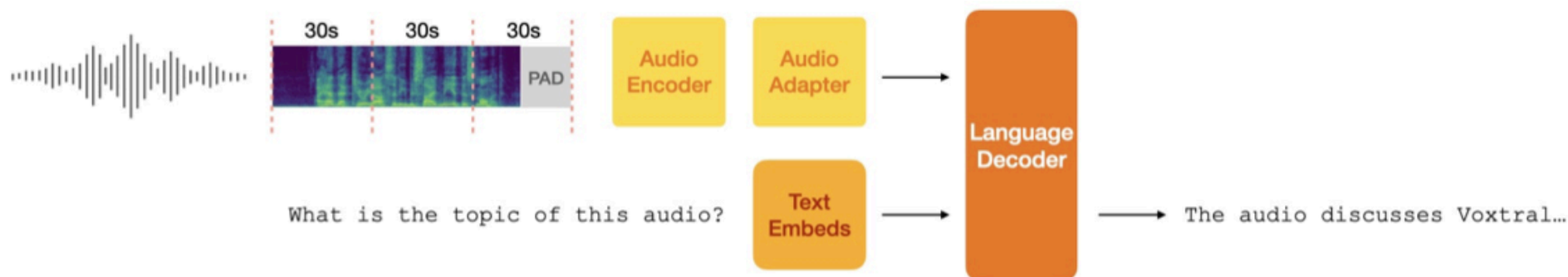
# dekodujemy samą odpowiedź
answer = processor.batch_decode(new_tokens, skip_special_tokens=True)[0]
```

Python

```
input_features.shape = (1, 128, 149)
answer = The original content of this audio is: 'a golden fortune and a happy life.'
```

Voxtral

Voxtral to rodzina modeli audio od Mistral. Obsługuje zarówno **chat z audio**, jak i **wyspecjalizowaną transkrypcję**. Pokazuje, jak łączy się moduł audio z nowoczesnym modelem językowym.



1. Audio trafia do **Whisper encoder**.
2. Powstają reprezentacje akustyczne.
3. **Multimodal projector** mapuje je do przestrzeni modelu językowego.
4. Model językowy generuje odpowiedź albo transkrypcję.

Voxtral

```
import torch, transformers

processor = transformers.AutoProcessor.from_pretrained("mistralai/Voxtral-Mini-3B-2507")
model = transformers.VoxtralForConditionalGeneration.from_pretrained(
    "mistralai/Voxtral-Mini-3B-2507"
).eval()

# prosimy o wejście do zadania ASR
inputs = processor.apply_transcription_request(
    audio="sample.wav", # wskazujemy plik audio
    language="en",      # podajemy język nagrania
    model_id="mistralai/Voxtral-Mini-3B-2507",
) # processor tworzy prompt i cechy audio

# wyłączamy gradienty
with torch.inference_mode():
    # model dopisuje odpowiedź
    generated_ids = model.generate(**inputs, max_new_tokens=64)

new_tokens = generated_ids[:, inputs["input_ids"].shape[1] :]
transcription = processor.batch_decode(new_tokens, skip_special_tokens=True)[0]
```

Python

```
input_features.shape = (1, 128, 3000)
transcription = A golden fortune and a happy life?
```

Podsumowanie

- Cecha to użyteczny, mierzalny opis sygnału.
- Skala melowa i MFCC kompresują informację częstotliwościową w sensowny sposób.
- Mowa ma strukturę czasową, więc reprezentacja musi zachować kolejność i przejścia.
- Embedding to nauczony wektor, w którym podobne dźwięki są blisko siebie.
- HuBERT uczy reprezentacji przez maskowanie i przewidywanie ukrytych jednostek.
- Whisper jest architekturą encoder-decoder mapującą log-mel na tekst.
- Modele językowe z modalnością audio łączą enkoder audio, projector i duży model językowy.